

j4_display_right

Pot-label display board for the Johnny 4 robot controller -- the landscape display on the RIGHT of the panel. It sits directly above four potentiometers and acts as an electronic label for them: the bottom of the screen reads **IRIS, COLOR, BRIGHTNESS, VOLUME** (left to right), each with a live value bar.

Since the controller's four ADS1115 ADCs have their channels allocated to the joysticks and face pots (with only spares left for the future 17th-19th pot readouts), this board carries the system's fifth ADS1115, which reads the four pots beneath it and immediately streams the values to the TTGO [j4_controller](#) over UART.

What this board does

- Reads 4 pots (IRIS, COLOR, BRIGHTNESS, VOLUME) via a dedicated ADS1115 on its own I2C bus
- Streams the raw readings to the TTGO controller at 25 Hz as an ASCII line: `P:<iris>,<color>,<brightness>,<volume>\n` (the steady stream doubles as this board's heartbeat)
- Renders the label strip and live value bars on a 3.5" ILI9488 TFT in landscape

Where the pot values go from there:

Pot	Path	Effect
IRIS	controller -> receiver -> PCA9685	270-deg iris servo
COLOR	controller -> receiver -> WS2812B strip	LED color: red -> orange -> yellow -> green -> blue -> violet -> white
BRIGHTNESS	controller -> receiver -> WS2812B strip	LED brightness, off to maximum
VOLUME	controller -> receiver -> j4_talk (Teensy)	audio output volume

Hardware

Component	Details
Microcontroller	SeeedStudio XIAO ESP32S3
Display	3.5" TFT LCD, 480x320, SPI, ILI9488 driver (landscape)
ADC	ADS1115, 16-bit, 4 channels, I2C addr 0x48 (ADDR pin to GND)

Pin assignments

XIAO Pin	GPIO	Function
3V3	n/a	Display VCC + ADS1115 VDD (3.3V power)
GND	n/a	Display/ADS1115 GND, shared with TTGO GND
D0	GPIO1	TFT backlight (HIGH = on)
D1	GPIO2	TFT RST

D2	GPIO3	TFT DC
D3	GPIO4	TFT CS
D4	GPIO5	I2C SDA (ADS1115)
D5	GPIO6	I2C SCL (ADS1115)
D6	GPIO43	UART TX to TTGO GPIO26
D7	GPIO44	UART RX from TTGO GPIO25 (face-preset messages)
D8	GPIO7	TFT SCLK
D9	GPIO8	TFT MISO
D10	GPIO9	TFT MOSI

Pin diagram

Physical layout, component side up, USB-C at the TOP. The two header rails read top-to-bottom as shown.

LEFT rail			RIGHT rail		
+--[USB-C]--+					
TFT BL -- D0 (GP1)			5V	(unused)	
TFT RST -- D1 (GP2)		XIA0	GND	ground	
TFT DC -- D2 (GP3)		ESP32-S3	3V3	TFT VCC + ADS1115 VDD	
TFT CS -- D3 (GP4)			D10 (GP9)	TFT MOSI	
I2C SDA -- D4 (GP5)			D9 (GP8)	TFT MISO	
I2C SCL -- D5 (GP6)			D8 (GP7)	TFT SCLK	
LINK TX -> TTGO 26 -- D6 (GP43)			D7 (GP44)	LINK RX <- TTGO 25	
+-----+					

D4/D5 are the XIA0's native I2C pins -- the ADS1115 hangs there (addr 0x48).
The TTGO->XIA0 direction (D7) carries the face-preset messages (M:/X:).

ADS1115 pin diagrams

The four pots under the screen, left to right as labeled on the display: IRIS, COLOR, BRIGHTNESS, VOLUME. Every pot's wiper goes to its A-pin; the outer legs go to 3.3V and GND (all four share the rails).

ADS1115 @ 0x48 (ADDR -> GND)			on the XIA0's own I2C bus		
+-----+					
VDD		XIA0 3V3			
GND		XIA0 GND			
SCL		XIA0 D5 (GPIO 6)			
SDA		XIA0 D4 (GPIO 5)			
ADDR		GND = address 0x48			
ALRT		not connected			
A0		IRIS pot wiper	->	270-deg iris servo (PCA9685 on j4_receiver)	
A1		COLOR pot wiper	->	WS2812B strip color (j4_receiver)	
A2		BRIGHTNESS pot wiper	->	WS2812B strip brightness (j4_receiver)	

```
| A3 | VOLUME pot wiper -> j4_talk audio volume
+-----+
```

Raw counts stream to the TTGO as `P:<iris>,<color>,<brightness>,<volume>` lines at 25 Hz; the controller scales them with its standard `processPot()`. The IRIS pot shares its function with `fader_right` on the controller (last-mover-wins, see `j4_controller`).

Wiring to TTGO controller

3.3V logic on both sides -- no level shifter needed.

XIAO	TTGO
D6 (GPIO43) TX	GPIO26 RX
D7 (GPIO44) RX	GPIO25 TX (face-preset messages)
GND	GND

Wiring to ILI9488 display module

XIAO	Display pin
3V3	VCC
GND	GND
D0 (GPIO1)	LED / BL (backlight)
D1 (GPIO2)	RST
D2 (GPIO3)	DC
D3 (GPIO4)	CS
D8 (GPIO7)	SCK
D9 (GPIO8)	MISO (SDO)
D10 (GPIO9)	MOSI (SDI)

Wiring to ADS1115

ADS1115	Connect to
VDD	XIAO 3V3
GND	XIAO GND
SCL	XIAO D5 (GPIO6)
SDA	XIAO D4 (GPIO5)
ADDR	GND (I2C address 0x48)
A0	IRIS pot wiper

A1	COLOR pot wiper
A2	BRIGHTNESS pot wiper
A3	VOLUME pot wiper

Each pot's outer legs go to 3.3V and GND (all four share the rails). The gain, data rate, and mode match the controller's own ADS1115s, so the raw counts scale identically (0 to ~17000 full-travel) and the controller can run its standard `processPot()` on them.

UART protocol

One ASCII line every 40 ms (25 Hz):

```
P:<iris>,<color>,<brightness>,<volume>\n
```

Values are raw ADS1115 counts. The controller scales them (`processPot()`) and forwards: iris/color/brightness/volume ride the ESP-NOW control packet to `j4_receiver`, which drives the iris servo and the WS2812B strip and relays volume to `j4_talk`. Any received `P:` line also marks this board CONNECTED on the controller's connection-status screen.

Downstream (TTGO to this board, on the previously-reserved direction), two line types drive the message band:

```
M:<line1>|<line2>\n    show a two-line message (face-preset save prompt, confirmations)
X:\n                  clear it
```

Anything else is treated as line noise from a floating RX pin and ignored; the buffer is length-capped, and a message self-clears after 15 seconds in case the `X:` gets lost. Message timing is owned by the controller, so this board stays dumb and never blocks on the link.

Display layout (480 x 320 landscape)

```
+-----+ y=0
|                KEVCO                | header (44px)
+-----+ green line
|                |
| message band (face-preset prompts)   |
|                |
+-----+ dim line
| [bar]  [bar]  [bar]  [bar]           | value bars (36px)
| IRIS   COLOR  BRIGHTNESS  VOLUME     | labels (44px)
+-----+ y=320
```

Each label cell is a quarter of the screen width, centered above its pot. If the image is upside down relative to the panel, change `setRotation(1)` to `setRotation(3)` in `main.cpp`.

Dependencies

Managed via PlatformIO (`platformio.ini`):

Library	Version
Bodmer/TFT_eSPI	latest from GitHub

TFT_eSPI is configured entirely through `build_flags` in `platformio.ini` -- no `User_Setup.h` file needed.

Building and uploading

```
# Build
pio run

# Upload
pio run --target upload

# Monitor serial output (USB CDC debug)
pio device monitor
```

If the upload fails, hold the **BOOT** button on the XIAO, tap **RESET**, then release BOOT to force download mode.

Related projects

- [j4_controller](#) -- the TTGO T-Display this board streams pot data to
- [j4_display_left](#) -- the portrait jukebox display on the left of the panel
- [j4_receiver](#) -- where the iris servo and WS2812B strip live